

Rapport — Jour 1

Étage régulier : automates finis & transducteurs

Binôme : DOSSOU Persévérance & AGBOGBA Silas • Modules : `formlang/dfa.py`, `nfa.py`, `fst.py`

A. Réponses théoriques

Q1.1 (0,5) — expression régulière

$$L_1 = (a|o|r)^* o r (a|o|r)^*$$

Justification : tout mot de L_1 s'écrit comme un préfixe quelconque sur $\{a, o, r\}$, suivi du facteur exact `or`, suivi d'un suffixe quelconque sur $\{a, o, r\}$. Comme l'alphabet est $\{a, o, r\}$, cette regex engendre exactement les mots contenant `or` comme facteur, et aucun mot n'en contenant pas (le facteur `or` doit apparaître littéralement entre les deux étoiles).

Q1.2 (1) — du non-déterminisme à l'AFD minimal

(a) **AFN.** États q_0, q_1, q_2 ; q_0 initial, q_2 final.

$$\delta(q_0, a) = \{q_0\}, \delta(q_0, o) = \{q_0, q_1\}, \delta(q_0, r) = \{q_0\}, \quad \delta(q_1, r) = \{q_2\}, \quad \delta(q_2, a) = \delta(q_2, o) = \delta(q_2, r) = \{q_2\}.$$

(b) **Table des sous-ensembles (déterminisation).**

état AFD	sur o	sur a,r	remarque
$S_0 = \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	initial
$S_1 = \{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0\}$ (a), $\{q_0, q_2\}$ (r)	o « armé »
$S_2 = \{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	accepteur
$S_3 = \{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$ (les deux)	accepteur

(c) **Partitions de minimisation (Moore).** Partition initiale : $\{S_0, S_1\}$ (non-finaux) | $\{S_2, S_3\}$ (finaux). Raffinement : sur **r**, $S_0 \rightarrow S_0$ (bloc non-final) mais $S_1 \rightarrow S_2$ (bloc final) $\Rightarrow S_0$ et S_1 se séparent. S_2 et S_3 mènent toujours au bloc final quel que soit le symbole $\Rightarrow$ ils fusionnent. Partition finale : $\{S_0\}, \{S_1\}, \{S_2, S_3\}$.

(d) **AFD minimal.** 3 états : $A = S_0, B = S_1, C = \{S_2, S_3\}$ — voir diagramme et table δ dans `jour1_regulier.tex` (section notions). Nombre d'états : 3.

Q1.3 (0,5) — facteur vs sous-séquence

$$L'_1 = (a|o|r)^* o (a|o|r)^* r (a|o|r)^*$$

Différence : L_1 exige que o et r soient **consécutifs** (facteur) ; L'_1 exige seulement qu'un o apparaisse **avant** un r , avec un nombre quelconque de lettres entre les deux (sous-séquence).

Inclusion $L_1 \subseteq L'_1$. Si w contient or comme facteur, alors w contient un o immédiatement suivi d'un r : c'est en particulier un cas particulier de « o suivi, pas forcément immédiatement, d'un r » (l'écart entre les deux peut être nul). Donc $w \in L'_1$.

Mot de $L'_1 \setminus L_1$. $w = oar$: contient o avant r (donc $w \in L'_1$) mais ne contient pas le facteur or (donc $w \notin L_1$).

B. Exercices de programmation

E1.1

DFA.run/accepts implémentés dans `formlang/dfa.py`; table δ de `_DFA_OR` (états A/B/C) remplie dans `apps/shield/detector.py` conformément à l'AFD minimal de Q1.2. *Test test_detector_or : passé.*

E1.2

DFA.minimize : raffinement de Moore sur l'automate complété (`_completed`), partition finaux/non-finaux puis raffinement par signature des blocs cibles jusqu'à stabilisation. *Test test_minimisation_3_etats : passé.*

E1.3

NFA.to_dfa : construction des sous-ensembles à partir de `_eps_closure/_move` fournis, exploration en largeur des parties atteignables. *Test test_nfa_to_dfa_equivalence : passé.*

E1.4

leet_fst (un état `q0`, final, table 4 : `a, 0 : o, 3 : e, 1 : i, 5 : s`, `identity_on_missing=True`) + SequentialFST.transduce lettre par lettre. *Tests test_leet_idempotent, test_fst_composition : passés.*

Sortie de référence obtenue

```
$ pytest -q tests/test_regular.py
6 passed
```

(Le 7^e test du fichier, `test_delimiters_pda`, relève du Jour 2 — hors périmètre de cette fiche.)